Graph traversal and connectivity solutions

A07-B01

The lists are A:[B,C], B:[A,D,E], C:[A,D,E], D:[B,C,F], E:[B,C,F], F:[D,E,G], G:[F], H:[]. In A..H order, the matrix rows are `01100000`, `10011000`, `10011000`, `01100100`, `01100100`, `00011010`, `00000100`, `00000000`. There are 18 list entries and 18 ones.

Directing the edges gives 9 entries. Adding two parallel A→B arcs and one B→B loop gives 12 entries. The binary matrix gains only the B→B relation, so it has 10 ones. Preserve A→B multiplicity 3 with a count or separate edge records.

A07-B02

Each row records a completed scan, except the initial row.

Scan	Newly discovered	Q, also the gray-vertex order	Black vertices
Initial	A	[A]	∅
A	B,C	[B,C]	A
B	D,E	[C,D,E]	A,B
C	∅	[D,E]	A,B,C
D	F	[E,F]	A,B,C,D
E	∅	[F]	A,B,C,D,E
F	G	[G]	A,B,C,D,E,F
G	∅	[]	A,B,C,D,E,F,G

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

In A..H order, d=(0,1,1,2,2,3,4,∞) and π=(NIL,A,A,B,B,D,F,NIL). The path is A,B,D,F,G. When C reaches D and E, both are gray. B already set their first-discovery records, so the algorithm does not update them. H stays white.

A07-B03

The dequeue order is A,C,B,E,D,F,G. Distances are (0,1,1,2,2,3,4,∞). Parents are (NIL,A,A,C,C,E,F,NIL). The parents of D, E, and F change. The path to G is A,C,E,F,G.

FIFO layer order does not depend on ordering within a layer. Induct on true distance k. The predecessor on a shortest path has distance k−1 and its scan ensures a label at most k. A parent path ensures that the label is no smaller than the true minimum. Every neighbor ordering therefore computes the same δ and may choose different parents among equal-length paths.

A07-B04

The initial queue contains only distance 0. Assume queue distances are nondecreasing and the tail is at most one larger than the head. Dequeue preserves this property. Scanning a vertex of distance k appends only k+1, while remaining queued distances lie between k and k+1. Enqueue therefore also preserves it.

Each parent edge increases d by 1, giving an actual length-d path and hence $\delta \leq d$. Induct on $\delta=k$. The preceding vertex u on a shortest path has label $k-1$ and is eventually scanned. If v is white, set $d[v]=k$. If v was discovered earlier, the earlier scan had distance at most $k-1$, giving $d[v] \leq k$. Thus $d=\delta$.

For LIFO, push a and then b, so b is popped first. The pop order is s,b,t,y,x,a. Vertex x gets $d[x]=4$ along s,b,t,y,x, although s,a,x has only 2 edges. When a eventually scans x, x is already discovered and its incorrect label stays unchanged.

A07-B05

From A, r=7 vertices are reachable. There are 7 enqueues, 18 list-entry scans, and $7 \times 8=56$ matrix probes. For the full forest there are 8 enqueues, still 18 list-entry scans, and $8 \times 8=64$ matrix probes. Both initialize 8 vertices once.

Single-source list time is $\Theta(n+\Sigma$ adjacency-list lengths of reachable vertices), bounded by $O(n+m)$. Matrix time is $\Theta(n+nr)$, with worst case $\Theta(n^2)$. Full-forest times are $\Theta(n+m)$ and $\Theta(n^2)$. Both require $O(n)$ extra space, excluding the input graph.

An isolated source has r=1. Its matrix BFS probes one row of n cells and initializes n vertices, for $\Theta(n)$ time. The matrix is already the input. Its original allocation is not a cost paid by every BFS call.

A07-B06

From s, the dequeue order is s,a,b,t. In s,a,b,c,t,z order, $d=(0,1,2,\infty,3,\infty)$ and $\pi=$ (NIL,s,a,NIL,b,NIL). The arc $c \to s$ cannot be traversed backward, so c is unreachable.

The outer loop starting with a finds {a,b,t}, then c finds {c,s}, then z gives {z}. These are not strongly connected components. The actual components are {a,b}, {c}, {s}, {t}, and {z}. For example, t cannot return to a.

In the weighted example, BFS chooses $s \to t$ with one edge and weight 10. The path $s \to x \to t$ has two edges and weight 2, which is the minimum total weight. BFS guarantees a minimum edge count, or minimum total weight when every edge has the same positive weight.

A07-D01

vertex	d	f	parent
A	1	12	NIL
B	2	9	A
C	3	4	B
D	10	11	A
E	5	8	B
F	6	7	E

t	event	stack after
1	discover A	A
2	discover B	AB
3	discover C	ABC
4	finish C	AB
5	discover E	ABE
6	discover F	ABEF
7	finish F	ABE
8	finish E	AB
9	finish B	A
10	discover D	AD
11	finish D	A
12	finish A	∅

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Only discovery and finish increment the clock; none of the nine scans does.

A07-D02

#	edge	target	type	stack	clock
1	AB	W	tree	A	1
2	BC	W	tree	AB	2
3	CA	G	back	ABC	3
4	BE	W	tree	AB	4
5	EF	W	tree	ABE	5
6	FE	G	back	ABEF	6
7	AC	B	forward	A	9
8	AD	W	tree	A	9
9	DC	B	cross	AD	10

A→C reaches a completed descendant. D and C have disjoint intervals, so D→C is cross. Tree and forward edges both satisfy strict nesting; parent information distinguishes them.

A07-D03

C→A→D is a path, but A is gray at d[C]=3. The white-path premise fails. D has parent A. The first interval pair is nested, making E a descendant of B. The second is disjoint. The last pair crosses, which would keep the later call active after the earlier call returned, violating the LIFO call rule.

A07-D04

After p discovers q, q scans back to gray p. The naive test reports a false cycle. In a simple graph skip v=parent[u]. In a multigraph skip only the exact edge ID used to enter u; a second parallel edge can form a length-2 cycle.

A07-D05

A back edge u→v closes the directed tree path v→…→u. Conversely choose earliest-discovered v on a cycle. The other cycle vertices are white at this snapshot. The cycle gives a white path to its predecessor u, making u a descendant of v. Thus u→v is back. If u=v, the self-loop proves the claim directly. Certificates here are A B C A and E F E.

A07-D06

DFS gives A D B E F C; Kahn gives A B D C E F.

edge	DFS positions	Kahn positions
AB	1<3	1<2
AC	1<6	1<4
AD	1<2	1<3
BC	3<6	2<4
BE	3<4	2<5
DC	2<6	3<4
EF	4<5	5<6

After restoring C→A, initial indegrees are A=1,B=1,C=3,D=1,E=1,F=1. There is no zero-indegree vertex. Kahn cannot start and leaves all six vertices, including vertices outside cycle A,B,C whose dependencies are blocked.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Connectivity solutions

A07-C01

A={a,b,c}, B={d,e}, C={f}, with A→B→C. DFS from a contains all vertices, but f cannot return to e.

A07-C02

a 1/12, b 2/11, c 3/10, d 4/9, e 5/8, f 6/7. Second pass: a,c,b | d,e | f.

A07-C03

If C is first, its first vertex reaches both white components and finishes last. If D is first, D cannot reach C and finishes before C is discovered. In the transpose, outgoing arcs from C lead only to larger-F components already visited. White mutual reachability inside C gives exactly C.

A07-C04

disc a,b,c=1,2,3. b→a gives low[b]=1. b returns but stays on S. c→b gives low[c]=2. At a, pop c,b,a. b is black but on S; a gray-only test would emit c incorrectly.

A07-C05

disc a..h=1..8; low=1,1,1,4,4,4,7,8. Cuts c,d; bridges cd,dg. Root a has one child. low[e]=disc[d]=4 makes de nonbridge, while nonroot d is a cut vertex.

A07-C06

fd,ef,de | dg | cd | ca,bc,ab; also record isolated h. Including de completes its triangle. Leaving it on the stack incorrectly merges it with the parent bridge cd.

A07-C07

Delete c: {a,b}|{d,e,f,g}|{h}, 3. Delete d: {a,b,c}|{e,f}|{g}|{h}, 4. Delete cd: {a,b,c}|{d,e,f,g}|{h}, 3. Delete de: {a,b,c,d,e,f,g}|{h}, 2.

A07-C08

Skipping every edge to the parent hides the second edge and leaves low[v]>disc[u]. Skip only the parent edge_id; the other edge gives equality. CLRS 22-2 partitions nonbridge edges. Here bridges are one-edge blocks and isolated vertices are recorded separately.

References and attribution

CLRS, Introduction to Algorithms, 3rd ed., § § 22.1–22.5 and Problem 22-2. Bridges are included here as one-edge blocks. Original handwritten notes 演算法_wjungle, PDF pp. 34–36, 40, for DFS notation and BFS augmenting paths. MIT 6.046J Fall 2004, L13 pp. 2–6 and L14 pp. 43–59; Fall 2005, lec16 pp. 2–5 and lec17 pp. 43–59. Representation and BFS background. MIT 6.006 Fall 2011 Lecture 14 pp. 2–6; Spring 2020 Lecture 10 pp. 2–4. DFS and topological order. Robert Tarjan, “Depth-First Search and Linear Graph Algorithms,” SIAM J. Comput. 1(2), 146–160 (1972), doi:10.1137/0201010. SCCs.

Original educational synthesis by Lin Cing. Original material is licensed under CC BY 4.0. The referenced textbook is not distributed with these notes.

https://github.com/xinhi6716/computer-science-self-study